```
; Define write and read macros for syscall I/O
%macro write 2
        mov rax, 1      ; syscall number for write
        mov rdi, 1      ; file descriptor 1 (stdout)
        mov rsi, %1     ; buffer to write
        mov rdx, %2     ; length of buffer
        syscall
%endmacro

%macro read 2
        mov rax, 0      ; syscall number for read
        mov rdi, 0      ; file descriptor 0 (stdin)
        mov rsi, %1     ; buffer to read into
        mov rdx, %2     ; number of bytes to read
        syscall
%endmacro

section .data
        ; Static data and messages
        dispMsg db "Write an X86/64 ALP to count
number of positive and negative numbers from the array",10,
"Name:- Sumaanyu",10,"Roll:- 7256",10,"Date of
Performance:- 17/03/2025",10
        dispMsglen equ $-dispMsg

        msg1 db "Number : "
        msg1len equ $-msg1

        endl db 10      ; newline
        pcount db 0     ; positive count
        ncount db 0     ; negative count

section .bss
        ; Uninitialized data
        hexnum resq 5     ; reserve space for 5 quadword
numbers (64-bit)
        num1 resb 17     ; input buffer for string number
        temp resb 16     ; temporary buffer for display

section .text
        global _start

_start:
        ; Display initial message
        write dispMsg, dispMsglen

        ; Prepare to read 5 numbers
        mov rcx, 5
        mov rsi, hexnum

nextnum:
        ; Save loop context
        push rcx
        push rsi

        ; Prompt for number
        write msg1, msg1len
        read num1, 17

        ; Convert ASCII input to hexadecimal in RBX
        call ascii_hex

        ; Restore context and store the number in array
        pop rsi
        pop rcx
        mov [rsi], rbx
        add rsi, 8

        ; Check if number is negative by testing sign bit
(bit 63)
        bt rbx, 63
        jnc pcnt        ; if not negative, go to positive count

        ; Increment negative counter
        inc byte[ncount]
```

```
        loop nextnum

pcnt:
        ; Increment positive counter
        inc byte[pcount]
        loop nextnum

        ; Reset rsi and rcx to display all numbers
        mov rsi, hexnum
        mov rcx, 5

next2:
        push rcx
        push rsi
        mov rbx, [rsi]    ; get each number
        call display      ; display it in hex
        pop rsi
        pop rcx
        add rsi, 8
        loop next2

        ; Display newline and positive count
        write endl, 1
        mov bl, [pcount]
        call display8

        ; Display newline and negative count
        write endl, 1
        mov bl, [ncount]
        call display8

        ; Exit
        mov rax, 60       ; syscall number for exit
        mov rsi, 0        ; status 0
        syscall

; --- Convert ASCII string to 64-bit hex in RBX ---
ascii_hex:
        mov rbx, 0
        mov rsi, num1
        mov rcx, 16       ; max digits to process

next:
        rol rbx, 4        ; rotate left 4 bits (shift nibble)
        mov al, [rsi]     ; get character

        cmp al, '9'
        jbe sub30h        ; digit <= 9

        sub al, 7h        ; if letter A-F, adjust ASCII

sub30h:
        sub al, 30h       ; convert ASCII to numeric value
        add bl, al        ; store in rbx
        inc rsi
        loop next
        ret

; --- Display 64-bit hex value stored in RBX ---
display:
        mov rsi, temp
        mov rcx, 16       ; 16 hex digits

next1:
        rol rbx, 4
        mov al, bl
        and al, 0Fh       ; get lower nibble
        cmp al, 9
        jbe add30h

        add al, 7h        ; adjust for A-F
```

```
add30h:
        add al, 30h       ; convert to ASCII
        mov [rsi], al
        inc rsi
        loop next1

        write temp, 16
        write endl, 1
        ret

; --- Display an 8-bit number (used for counts) ---
display8:
        mov rsi, temp
        mov rcx, 2        ; 2 hex digits (8 bits)

next18:
        rol bl, 4
        mov al, bl
        and al, 0Fh
        cmp al, 9
        jbe add30h8

        add al, 7h

add30h8:
        add al, 30h
        mov [rsi], al
        inc rsi
        loop next18

        write temp, 2
        write endl, 1
        ret
```